

OBL2-OS

1. Processes and threads

1. The difference between a process and a thread

The difference between a process and a thread is that a process is a program in execution with its own isolated address space and resources; a thread is the unit of execution (scheduling) within a process. A thread can be defined as two things: a logical processor, which is the actual hardware threads in your CPU, and a software thread in the sense of parallel programming, where a process spawns subthreads to do processing in parallel. Threads in the same process share the address space, heap, and open files, but each has its own stack and registers - so logic in thread A doesn't block thread B. Processes, by contrast, are isolated from each other and must use IPC to communicate.

2. Scenarios where it would be desirable to use: multithreading, and multiprocessing

Multithreading

A scenario where you would want to use multithreading is in heavy load videogames. A videogame's performance would always benefit from multithreading, even at the cost of added complexity, as videogames use independent logic for subsystems such as: rendering, game updates, packet handling, entity processing, etc. All of these systems' are largely segregated, especially at the core level, but still require a shared state for the game to function - hence why multithreading is preferred.

Multiprocessing

A scenario where multiprocessing is preferable is computational science, such as physics simulations or distributed machine-learning training. The workload is CPU-bound and split into largely independent tasks that operate on separate data, so there is little need for a shared address space - the IPC cost of passing results between processes is small compared to the parallel speedup. Multiprocessing also lets the work scale across multiple cores or machines, and fault isolation means one crashed worker process does not take down the whole computation.

The same isolation argument applies to robotics and services: in a robot, a crashed sensor-driver process must not bring down the motor controller, so safety-critical components run as separate processes (as in ROS 2). Similarly, a web server can fork a process per request or run a pool of worker processes, so a crash or memory leak in one request does not kill the server.

Rule of thumb

The rule of thumb when deciding between multithreading and multiprocessing is: use threads when tasks need cheap communication through shared state; use processes when tasks are independent, CPU-bound, or need fault/security isolation.

3. Why each thread requires a thread control block

Each thread is an independent unit of scheduling and execution, so the OS must be able to save, track, and restore each thread's state separately. The TCB therefore holds the state that is private to each thread: its CPU context (program counter, stack pointer, and registers), its scheduling state (running/ready/blocked and priority), its stack location, and per-thread kernel data such as signal masks and thread ID. On a context switch, the scheduler saves the outgoing thread's registers into its TCB and restores the incoming thread's from its own - without per-thread TCBs, threads would overwrite each other's execution state, and the OS could not schedule, block, or wake threads individually.

The process's shared resources - address space, open files, and heap - remain in the PCB, which points to a list of TCBs, one per thread. This split mirrors the distinction from earlier: threads share the address space but own their execution state privately, and the TCB is precisely that private state.

```
PCB --> address space, open files, UID, ...  
+---> TCB [thread A: registers, PC, SP, state, priority]  
+---> TCB [thread B: registers, PC, SP, state, priority]
```

4. The difference between cooperative threading and pre-emptive threading

The difference is who decides when a thread gives up the CPU. In cooperative threading, a thread runs until it voluntarily yields - via a `yield()` call or by blocking on I/O or a lock - so switches only happen at points the thread itself chooses. In pre-emptive threading, the OS can forcibly switch threads at any time, typically when a timer interrupt signals that a thread's time slice, also known as quantum, has expired or a higher-priority thread becomes ready.

A cooperative context switch is simple: the thread calls `yield()`, its context (registers, PC, SP) is saved to its TCB, the scheduler picks the next ready thread, and that thread's context is restored from its TCB. A pre-emptive switch starts with a timer interrupt: the hardware traps to the kernel and saves the running thread's context, the scheduler decides to switch, the outgoing thread's state is stored in its TCB and it is returned to the ready queue, and the incoming thread's context is restored. Because a pre-emptive switch can occur at any instruction boundary - possibly mid-way through updating shared data - pre-emptive systems require explicit synchronization, whereas cooperative switching is cheaper but lets one misbehaving thread starve all others.

2. C program with POSIX threads

1. Which part of the code is executed when a thread runs

The function `go ( void *go(void *n) )`. Each thread starts executing at the top of `go` with its own argument `n` (0–9, passed as the fourth argument to `pthread_create`). The function prints `Hello from thread <n>`, then terminates the thread with `pthread_exit(100 + n)`, delivering `100 + n` as the thread's exit value to whichever thread joins it.

2. Why the order of the "Hello from thread X" messages change each run

Because the ten threads are scheduled independently by the OS scheduler, and creation order does not guarantee execution order. `pthread_create` only makes a thread ready; when each thread actually runs depends on the scheduler, core availability, and system load. The `printf` calls are unsynchronized, so the interleaving is a race and the output order is nondeterministic. Only the "Thread X returned" lines are always in order, because `main` joins the threads strictly in order 0–9.

3. Minimum and maximum number of threads when thread 8 prints "Hello"

Minimum: 2 - `main` plus thread 8 itself. All nine other threads may already have printed, called `pthread_exit`, and been fully terminated before thread 8 gets scheduled. Maximum: 11 - `main` plus all ten worker threads alive simultaneously, since all are created before any join and thread 8 may print while every other thread still exists (running or ready).

4. The use of `pthread_join`

`pthread_join(threads[i], (void*)&exitValue)` makes the calling thread (`main`) block until thread `threads[i]` terminates. It serves three purposes here: firstly, synchronization - it guarantees `main` does not finish, and the process does not exit, before all workers have completed, which would otherwise kill threads that have not run yet; secondly, retrieval - the second parameter receives the value the thread passed to `pthread_exit ( 100 + n )`, which is printed as

Thread `i` returned with `100+i` ; thirdly, reclamation - it lets the system free the terminated thread's resources (stack, TCB); a thread that terminates without being joined lingers as a "zombie" thread.

5. What would happen if `go` were changed as shown

`pthread_exit(100 + n)` is still reached for every thread, including thread 5 - `sleep(2)` only blocks thread 5 for two seconds and then returns; it does not terminate the thread. Thread 5's flow becomes: `print` → `sleep 2s` → `pthread_exit(105)` . Since `main` joins in order 0–9, it blocks inside `pthread_join(threads[5], ...)` during those two seconds, delaying the whole program's completion by ~2 seconds even though threads 6–9 may have exited long before.

6. When `pthread_join` returns, in what state is thread X?

Thread X is in the terminated - also known as dead - state: it has stopped executing, its exit value has been delivered, and its resources (stack, TCB) have been reclaimed. It is no longer a schedulable entity - not running, ready, or blocked - and its thread ID can no longer be joined or signaled.